SMART CONTRACT SECURITY ASSESSMENT

MegaTao

Decentralized Perpetual Futures Exchange on Bittensor EVM

AUDIT PERIOD	January 2026 — February 2026
REPORT DATE	February 26, 2026
VERSION	v1.0-final
STATUS	All findings resolved or acknowledged
PREPARED FOR	MegaTao
PREPARED BY	Bitsec — AI-Powered Smart Contract Security
CONFIDENTIALITY	Public Release
COMMIT HASH	`9e473a7fa070a39650ed74250b915ec75145f1d9`

FINDINGS SUMMARY

2	4	6	2	13
Critical	High	Medium	Low	Info

© 2026 Bitsec. Redistribution permitted in unmodified form with attribution.

Table of Contents

Section 0 — Notices & Legal

Section 1 — About Bitsec

Section 2 — Engagement Overview

Section 3 — Protocol Summary

Section 4 — Methodology

Section 5 — Executive Summary

Section 6 — Codebase Quality Assessment

Section 7 — Detailed Findings

- Critical: BIT-MEGA-001, BIT-MEGA-002
- High: BIT-MEGA-003 through BIT-MEGA-006
- Medium: BIT-MEGA-007 through BIT-MEGA-012
- Low: BIT-MEGA-013, BIT-MEGA-014
- Informational: BIT-MEGA-015 through BIT-MEGA-027

Section 8 — Gas Optimization Findings

Section 9 — Fix Review / Remediation Log

Section 10 — Appendices

Section 0 — Notices & Legal

Copyright

This report is the property of Bitsec. Redistribution is permitted in unmodified form with attribution.

Coverage Disclaimer

- This assessment is time-boxed to the engagement period (January–February 2026).
- Findings are not exhaustive. The absence of a reported vulnerability does not guarantee its absence in the codebase.
- Bitsec's AI agent methodology augments but does not replace manual human judgment. All agent-generated findings were triaged and validated by a senior auditor.
- This report reflects the state of the code at commit `9e473a7`. Subsequent changes are not covered.
- No formal verification or exhaustive symbolic execution was performed.
- This report does not constitute investment advice, legal advice, or an endorsement of the MegaTao protocol.

Canonical Source

The authentic version of this report is published on the Bitsec audit reports repository (github.com/Bitsec-AI/audit-reports). Any version obtained from other sources should be verified against the canonical copy.

Section 1 — About Bitsec

Bitsec is an AI-powered smart contract security firm operating on Bittensor Subnet 60. Our methodology leverages competitive AI agent mining: multiple independent agents analyze target codebases in parallel, and findings are deduplicated, scored, and validated by John Yu. This approach combines the breadth of automated analysis with the precision of expert judgment. As agents improve from competitive subnet dynamics, we offer ongoing security monitoring.

Methodology

AI agents compete on Subnet 60 to discover vulnerabilities. Agents are scored using an F2-weighted metric (recall-favored over precision), calibrated against the SCA-Bench smart contract audit benchmark. The competitive structure incentivizes agents to find real, exploitable vulnerabilities.

Contact

john@bitsec.ai | bitsec.ai

Section 2 — Engagement Overview

2.1 Engagement Details

Field	Value
Client	MegaTao
Audit type	Full Audit
Engagement start	2026-01-13
Report delivered	2026-02-26
Audit lead	John Yu
Agent pool	Bitsec Subnet 60 miners (9 agents)

2.2 Scope

Repository: `megatao-monorepo` at commit `9e473a7fa070a39650ed74250b915ec75145f1d9`

Language / Framework: Solidity 0.8.28, Foundry, EVM target london (pre-Paris, Bittensor compatibility)

In-scope contracts (9,520 LOC):

File	LOC
src/modules/Vault.sol	910
src/modules/Position.sol	896
src/modules/Orderbook.sol	756
src/libraries/ExecutionLib.sol	667
src/storage/Storage.sol	616
src/libraries/VaultOperationsLib.sol	538
src/libraries/OrderbookLib.sol	514
src/libraries/PositionLib.sol	504
src/libraries/VaultLib.sol	479
src/modules/Admin.sol	471
src/modules/Liquidation.sol	417
src/modules/Funding.sol	393

File	LOC
src/libraries/FundingLib.sol	289
src/modules/OrderbookQueries.sol	276
src/modules/Trading.sol	269
src/modules/Margin.sol	263
src/modules/Staking.sol	223
src/libraries/Constants.sol	157
src/modules/PositionQueries.sol	117
src/libraries/LiquidationLib.sol	104
src/libraries/SlippageLib.sol	101
src/libraries/PriceLib.sol	61
src/FuturesProxy.sol	55
src/libraries/TokenOps.sol	54
src/libraries/ValidationLib.sol	47
src/Futures.sol	43
src/utils/Fallback.sol	37
src/implementations/*.sol (6 files)	143
src/modules/wrappers/*.sol (8 files)	120

Out of scope:

- `src/mocks/` — Test mock contracts
- `src/interfaces/` — Interface definitions (2,014 LOC)
- `test/` — Test suite (56,257 LOC across 120 files)
- `script/` — Deployment scripts
- `lib/` — OpenZeppelin and other vendored dependencies
- `web/`, `sdk/` — Frontend and SDK code

Prior audits: This is the second external security assessment of the MegaTao protocol.

2.3 Objectives

The engagement focused on:

- **Core trading logic:** Position open/close, margin accounting, PnL settlement
- **Liquidation correctness:** Margin ratio calculations, partial/full liquidation, keeper incentives, bad debt handling
- **Vault security:** Exposure tracking, counterparty risk, reserve management, circuit breakers
- **Oracle integrity:** On-chain pool price derivation, manipulation resistance
- **Access control:** Role-based permissions, upgrade authorization, admin key risk
- **Money flow paths:** Deposit/withdrawal, fee collection, reward distribution, reentrancy
- **Cross-contract interactions:** Delegatecall chain integrity, self-call patterns, state consistency

Client-provided invariants:

- Vault reserves must never underflow
- Positions cannot be opened and liquidated in the same block (flash loan protection)
- Insurance fund is consumed before vault reserves on bad debt
- Aggregate vault exposure is bounded by configurable caps

Section 3 — Protocol Summary

What MegaTao Does

MegaTao is a decentralized perpetual futures exchange built on the Bittensor EVM (Finney mainnet). Traders deposit TAO (native or wrapped ERC20) as collateral and open leveraged long or short positions on Bittensor subnet token prices (TAO/Alpha pairs). The protocol supports up to 30x leverage with a default of 3x.

The system uses a hybrid execution model: incoming market orders first match against resting limit orders on a peer-to-peer orderbook, then route any unfilled remainder to a protocol-owned vault that acts as the central counterparty. This ensures guaranteed execution at the cost of slippage, which is dynamically computed based on vault utilization and order size.

Positions accrue two separate ongoing fees: (1) a **funding rate** based on open interest imbalance between longs and shorts (Binance-style linear model, capped at 0.75% per 8 hours), and (2) an **interest fee** based on vault mark-to-market utilization (quadratic model: `20% × utilization² + 0.01%`). These fees maintain market balance and compensate the vault for counterparty risk.

Key Components

Component	Role
FuturesProxy	UUPS proxy entry point; delegates all calls to Futures
Futures / TradingOperationsImpl	Top-level implementation; routes to Trading, Margin, Vault
Trading	User-facing order entry; coordinates funding updates and order execution
Orderbook / OrderbookLib	Limit order storage, matching engine, P2P fill logic
Position / PositionLib	Position lifecycle: open, modify, close, PnL calculation
Vault / VaultLib / VaultOperationsLib	Protocol counterparty; exposure tracking, reserves, circuit breakers
Liquidation / LiquidationLib	Margin ratio checks, partial/full liquidation, keeper rewards, bad debt
Funding / FundingLib	Funding rate and interest fee calculation and settlement
Margin	Deposit/withdrawal, available margin computation
Admin	Market management, pause controls, fee withdrawal, upgrade auth
Staking	Native TAO validator staking via Bittensor precompile
Storage	ERC-7201 namespaced storage layout for all protocol state

Trust Model

Role	Capabilities
DEFAULT_ADMIN_ROLE	Grant/revoke all other roles
UPGRADER_ROLE	Authorize UUPS implementation upgrades
EMERGENCY_ROLE	Pause/unpause protocol and individual markets
MARKET_MANAGER_ROLE	Add/update markets
TREASURY_ROLE	Vault deposits/withdrawals, fee collection, staking operations
RISK_MANAGER_ROLE	Adjust deposit caps, exposure limits, bad debt thresholds
KEEPER_ROLE	Trigger funding rate updates

All roles are initially granted to the deployer. The protocol uses a multisig for admin roles but has no on-chain timelock.

External Dependencies

- **Price Oracle:** `IAlphaPriceOracle.calculatePriceFromReserves(netUid)` — derives prices from on-chain Bittensor subnet TAO/Alpha pool reserves. Single oracle source; no fallback.
- **Bittensor Staking Precompile:** `0x...0805` — native TAO staking to validators (native mode only).
- **IWETH9:** Wrapped TAO for ERC20 mode deposit/withdrawal.
- **OpenZeppelin Upgradeable:** AccessControl, ReentrancyGuard, Pausable, UUPS, ERC1967Proxy, Multicall.

Known Design Tradeoffs

- Single on-chain oracle with no TWAP or circuit breaker (no alternative oracle exists for Bittensor subnet prices)
- Shared insurance fund across all markets (deliberate: per-market fragmentation deemed less capital efficient)
- No timelock on admin functions (multisig provides operational delay)
- No liquidation cooldown (prioritizes exchange health, slowing liquidation increases bad debt during crashes)
- Keeper rewards paid regardless of position solvency (incentivizes liquidation when most needed)

Section 4 — Methodology

4.1 AI Agent Competition

Nine independent AI agents on Bitsec Subnet 60 analyzed the MegaTao codebase in parallel for hundreds of iterations. Each agent performed static analysis, symbolic reasoning, and LLM-guided vulnerability discovery. These potential findings undergo a triage, deduplication, and POC generation process which are validated by our lead auditor and shared with the MegaTao team.

The competitive structure produced 38 raw findings.

4.2 Finding Aggregation

- Raw agent findings were deduplicated based on root cause analysis
- Each finding was tagged with an **agent consensus count** indicating how many agents independently identified the same issue
- Findings were severity-classified using the Bitsec severity taxonomy (see Appendix B)
- Lead auditor (John Yu) validated all findings, wrote proof-of-concept exploits for critical/high findings, and triaged with the MegaTao team

4.3 Tooling

Tool	Usage
Slither	Static analysis, inheritance checks, reentrancy detection
Foundry	Fuzz testing, PoC exploit development
Manual review	Architecture analysis, business logic, cross-contract flows

Test suite assessment: The project includes 120 test files (56,257 LOC) with comprehensive coverage of core trading, liquidation, and vault functionality. Invariant tests are present for key protocol properties.

Section 5 — Executive Summary

5.1 Overall Assessment

MegaTao's codebase demonstrates strong engineering quality with well-structured modular architecture, comprehensive test coverage, and appropriate use of security primitives (reentrancy guards, access control, pausability, circuit breakers). The protocol's hybrid orderbook-vault execution model is sound and the ERC-7201 namespaced storage pattern correctly prevents upgrade storage collisions.

The audit identified two critical vulnerabilities (interest excluded from liquidation margin ratio; vault exposure asymmetry from P2P fills) that could have led to protocol insolvency. Both were identified by Bitsec and rapidly fixed before production deployment. The remaining findings center on oracle design tradeoffs inherent to operating on Bittensor L1, where no alternative price feed exists and no MEV infrastructure is currently deployed. These risks are acknowledged by the MegaTao team and mitigated by circuit breakers and operational controls.

Key positives:

- Robust test suite (56K+ LOC, invariant tests present)
- Correct reentrancy protection on all state-modifying external calls
- Defense-in-depth: circuit breakers, bad debt auto-pause, per-market exposure caps
- Clean separation of concerns via library pattern

Key concerns:

- Single oracle dependency, mitigated by governance controls (residual risk if MEV, flash loans, or concentrated alpha token liquidity pools enabled on Bittensor)
- No on-chain timelock for admin operations (relies on multisig)

5.2 Summary of Findings

ID	Title	Severity	Type	Status
BIT-MEGA-001	Interest Not Included in Liquidation Margin Ratio	Critical	Arithmetic	Resolved
BIT-MEGA-002	Vault Exposure Asymmetry on P2P Fill Reduction	Critical	Logic	Resolved
BIT-MEGA-003	Keeper Reward Paid on Insolvent Positions	High	Logic	Acknowledged
BIT-MEGA-004	Flash Crash Liquidation via Oracle Manipulation	High	Oracle	Acknowledged
BIT-MEGA-005	Maximum Value Extraction Attack Chain	High	Logic	Acknowledged
BIT-MEGA-006	Protocol Insolvency Attack Chain	High	Logic	Resolved
BIT-MEGA-007	No Admin Function Timelock	Medium	Centralization	Acknowledged
BIT-MEGA-008	Single Oracle Source with No Fallback	Medium	Oracle	Acknowledged

ID	Title	Severity	Type	Status
BIT-MEGA-009	Oracle Front-Running via Price Observation	Medium	MEV	Acknowledged
BIT-MEGA-010	Insurance Fund Not Isolated Per Market	Medium	Logic	Acknowledged
BIT-MEGA-011	Bad Debt Socialization to Vault Depositors	Medium	Logic	Acknowledged
BIT-MEGA-012	No Partial Liquidation Cooldown	Medium	DoS	Acknowledged
BIT-MEGA-013	Funding Rate Bounded Manipulation	Low	Oracle	Acknowledged
BIT-MEGA-014	Utilization Metric Excludes Unrealized PnL	Low	Logic	Acknowledged
BIT-MEGA-015	Stale Price Rejection	Informational	Validation	Resolved
BIT-MEGA-016	Oracle Heartbeat Check	Informational	Validation	Resolved
BIT-MEGA-017	Funding Update Race Condition	Informational	Logic	Resolved
BIT-MEGA-018	Concurrent Liquidation Double-Spend	Informational	Reentrancy	Resolved
BIT-MEGA-019	Interest Forces Premature Insolvency	Informational	Arithmetic	Resolved
BIT-MEGA-020	Double Exposure Tracking on P2P	Informational	Logic	Resolved
BIT-MEGA-021	Insurance Fund Bypass Path	Informational	Logic	Resolved
BIT-MEGA-022	Circuit Breaker Threshold Configuration	Informational	Logic	Resolved
BIT-MEGA-023	Dynamic Vault Capacity Calculation	Informational	Logic	Resolved
BIT-MEGA-024	Missing Vault Withdrawal Function	Informational	Logic	Resolved
BIT-MEGA-025	Missing Fee Withdrawal Mechanism	Informational	Logic	Resolved
BIT-MEGA-026	Native Transfer Reentrancy	Informational	Reentrancy	Resolved
BIT-MEGA-027	Unchecked Fee Deduction Underflow	Informational	Arithmetic	Resolved

5.3 Finding Distribution

Severity	Count	Resolved	Acknowledged	Not Applicable
Critical	2	2	0	0
High	4	1	3	0
Medium	6	0	6	0
Low	2	0	2	0
Informational	13	13	0	0
Total	**27**	**16**	**11**	**0**

Note: An additional 11 raw findings from agents were reclassified as "Not Vulnerable" during triage (incorrect assumptions about code behavior) and are excluded from this report. Total raw findings: 27 consolidated + 11 reclassified = 38. See Section 9 for the full reconciliation.

Status definitions:

- **Resolved** — The finding has been fixed by the development team and verified by the auditor. The root cause has been addressed in the codebase.
- **Acknowledged (Acceptable Risk)** — The development team has reviewed the finding and determined that the identified risk is acceptable given the protocol's design constraints, operational context, or existing mitigations.
- **Not Applicable** — The finding was determined to be invalid after further analysis.

Section 6 — Codebase Quality Assessment

6.1 Code Quality

Readability: Good. The modular separation between core logic (modules), pure computation (libraries), and storage is clean and consistent. Function names are descriptive and the delegatecall fallback chain, while complex, is well-documented in deployment scripts.

NatSpec coverage: Good. Interfaces have full NatSpec documentation. Implementation contracts have inline comments but inconsistent NatSpec on external functions.

Event emission: Comprehensive. All state-changing operations emit events. Liquidation, position, and vault events include sufficient indexed parameters for off-chain indexing.

Error handling: Strong. Custom errors are used throughout (gas-efficient). Revert messages are descriptive. No use of `assert()` for non-invariant checks.

6.2 Test Coverage

Test suite: 120 test files, 56,257 LOC. This represents a 5.9:1 test-to-source ratio, which is excellent.

Coverage areas:

- Unit tests for all library functions
- Integration tests for full trading flows (open, close, liquidate)
- Fuzz tests for margin calculations and slippage model
- Invariant tests for vault reserve consistency
- Edge case tests for flash loan protection, self-liquidation prevention, circuit breakers

6.3 Architecture Assessment

Contract structure: The multi-level delegatecall chain (`Futures` → `PositionImpl` → `OrderExecutionImpl` → `MarginVaultImpl` → `RiskManagementImpl` → `ViewsImpl`) is an effective approach to the 24KB contract size limit. The `Fallback.sol` utility cleanly chains modules without selector conflicts.

Upgradeability: UUPS pattern with ERC-7201 namespaced storage. The `_authorizeUpgrade` function validates that all six implementation identity selectors match between old and new implementations, preventing partial chain upgrades. However, immutable constructor parameters (`collateralToken`, `priceOracle`, `collateralScale`) are not validated during upgrades — a new implementation must be deployed with identical constructor arguments or behavior diverges silently.

Centralization risks: All admin roles initially granted to deployer. No on-chain timelock. The multisig provides operational delay but is not enforced at the smart contract level. `UPGRADER_ROLE` can replace the entire implementation, including modifying access control logic.

Attack surface: The primary attack surface is the oracle (single price source from on-chain pool reserves) and the vault (counterparty to all trades). Both are protected by circuit breakers and exposure caps.

6.4 Integration & External Dependency Risk

Oracle dependency: The protocol derives all prices from `IAlphaPriceOracle.calculatePriceFromReserves()`, which reads Bittensor subnet TAO/Alpha pool reserves. This is a live-computed view function, not a pushed oracle feed. It cannot be stale but can theoretically be manipulated via large pool swaps. No alternative oracle exists for Bittensor subnet prices.

Bittensor precompile: The staking precompile at `0x...0805` is a chain-native component. Risk is limited to the native TAO staking mode and is constrained by rate limits (~72-minute cooldown between operations per hotkey).

OpenZeppelin: Standard audited library versions. No custom modifications detected.

Section 7 — Detailed Findings

Critical

[BIT-MEGA-001] — Interest Not Included in Liquidation Margin Ratio

Severity	**Critical**
Type	Arithmetic
Target	`Liquidation.sol` → `getLiquidationStatus()`; `LiquidationLib.sol` → `calculateMarginRatio()`
Status	**Resolved**

Description

The liquidation margin ratio calculation did not include accrued interest payments. `LiquidationLib.calculateMarginRatio` computed PnL using only realized PnL and funding payments, omitting the interest fee that positions accrue based on vault utilization. This meant positions with significant accumulated interest debt appeared healthier than they actually were.

Impact

Positions with high interest debt could avoid liquidation even when their effective equity was below maintenance margin. In extreme cases, these positions would become insolvent (negative equity) before triggering liquidation, creating bad debt that must be absorbed by the insurance fund and vault reserves. At scale, this could lead to protocol insolvency.

Proof of Concept

```
// 1. Open highly leveraged position (30x) on a market with high vault utilization
// 2. Interest accrues at quadratic rate (20% * utilization^2 + 0.01%)
// 3. Position margin erodes from interest but margin ratio excludes this cost
// 4. Position equity drops below zero before liquidation triggers
// 5. Liquidation now produces bad debt absorbed by insurance/reserves
```

Recommendations

- **Short term:** Include `interestPayment` in `LiquidationLib.calculateMarginRatio`. **(Implemented)**
- **Long term:** Consider adding interest accrual to the real-time margin query functions used by the frontend.

Client Response

Fixed. `Liquidation.sol:66-78` *now calculates* `interestPayment` *and passes it to* `LiquidationLib.calculateMarginRatio`. `LiquidationLib.sol:91` *includes interest in the total PnL calculation.*

[BIT-MEGA-002] — Vault Exposure Asymmetry on P2P Fill Reduction

Severity	**Critical**
Type	Logic

Target	`VaultLib.sol → adjustExposureMappingsOnReduce(); VaultOperationsLib.sol → reduceVaultExposure()`
Status	**Resolved**

Description

When a position was partially filled via the P2P orderbook (not the vault), the vault's directional exposure tracking could become corrupted on position close. `adjustExposureMappingsOnReduce` did not clamp the reduction amount to the actual directional exposure, allowing P2P-filled portions to inflate the reduction beyond what the vault was actually exposed to.

Impact

Repeated exploitation could drain vault reserves by creating phantom collateral releases. Each P2P-filled position close would free collateral the vault never actually locked, steadily depleting `availableReserves` until the vault becomes undercollateralized against its real exposure.

Proof of Concept

```
// 1. Open large position that is partially filled via P2P orderbook
// (e.g., 60% P2P, 40% vault)
// 2. Close position -- vault exposure reduction applied to full position size
// 3. Vault releases collateral for 100% of position, but only locked 40%
// 4. Net effect: 60% of position notional released as phantom collateral
// 5. Repeat to drain vault reserves
```

Recommendations

- **Short term:** Clamp reduction to directional exposure in `adjustExposureMappingsOnReduce`. **(Implemented)**
- **Long term:** Add invariant tests that assert `sum(lockedCollateral) <= availableReserves` after every state transition.

Client Response

Fixed. `VaultLib.adjustExposureMappingsOnReduce` *(line 242-277) now clamps reduction to directional exposure.* `VaultOperationsLib.reduceVaultExposure` *(line 140-143) uses* `actualReduction` *for collateral release.*

High

[BIT-MEGA-003] — Keeper Reward Paid on Insolvent Positions

Severity	**High**
Type	Logic
Target	`Liquidation.sol:185 → keeper reward calculation`
Status	**Acknowledged (Acceptable Risk)**

Description

The keeper reward is calculated as 5% of the liquidated margin (`LIQUIDATION_FEE = 500 bps`) regardless of whether the position has positive equity. When a position is insolvent (negative equity), the keeper reward is still paid from the margin, effectively increasing the bad debt.

Impact

During mass liquidation events (market crash), many positions may be insolvent simultaneously. Each insolvent liquidation produces bad debt amplified by the 5% keeper fee. In a scenario with 1,000 TAO of insolvent positions, the keeper fee adds 50 TAO of additional bad debt.

Proof of Concept

```
// 1. Market crashes rapidly, many positions go underwater
// 2. Keeper liquidates insolvent position with 100 TAO margin, -150 TAO equity
// 3. Keeper receives 5 TAO reward from the 100 TAO margin
// 4. Remaining 95 TAO margin partially covers -150 TAO loss
// 5. Bad debt = 150 - 95 = 55 TAO (instead of 50 TAO without fee)
// 6. Keeper fee added 5 TAO of bad debt per liquidation
```

Recommendations

- **Short term:** Cap keeper reward at `min(reward, max(0, positionEquity))` so insolvent positions pay zero reward.
- **Long term:** Consider a tiered keeper reward that decreases as position health worsens.

Client Response

Acknowledged as acceptable risk. The 5% keeper fee is the standard DeFi mechanism to incentivize liquidation (GMX uses similar). Capping rewards would disincentivize liquidation during mass liquidation events — the exact moments when rapid liquidation is most critical to prevent cascading bad debt. The fee is deducted from the trader's margin, not the insurance fund directly; insurance is only touched when margin is insufficient.

[BIT-MEGA-004] — Flash Crash Liquidation via Oracle Manipulation

Severity	**High**
Type	Oracle Manipulation
Target	`Liquidation.sol:154 → priceOracle.calculatePriceFromReserves()`
Status	**Acknowledged (Acceptable Risk)**

Description

The protocol uses a single on-chain oracle that computes prices directly from Bittensor subnet pool reserves (TAO/Alpha ratio) with no TWAP validation, price deviation limits, or circuit breakers at the oracle level. An attacker with sufficient capital could manipulate pool reserves via large swaps to trigger mass liquidations at artificial prices.

Impact

An attacker could: (1) execute a large swap to manipulate the TAO/Alpha ratio, (2) trigger liquidations at the manipulated price, (3) reverse the swap, profiting from liquidated positions. Currently mitigated by absence of MEV infrastructure on Bittensor L1 and maxSlippage caps (10%).

Recommendations

- **Short term:** Implement a price deviation check: reject liquidations when current price deviates more than X% from a moving average.
- **Long term:** Implement a TWAP oracle or multi-block price aggregation when Bittensor infrastructure matures.

Client Response

Acknowledged as acceptable risk. The oracle computes prices LIVE from on-chain Bittensor subnet pool reserves. It is a view function that reads current chain state — not an externally pushed price. Traditional TWAP/staleness checks apply to Chainlink-style pushed feeds, not on-chain pool ratios. No alternative oracle exists for Bittensor subnet prices. No MEV infrastructure exists on Bittensor L1.

Bitsec note: The "no MEV on Bittensor" argument is valid today but fragile. We recommend the MegaTao team monitor for MEV infrastructure development on Bittensor and implement oracle bounds proactively.

[BIT-MEGA-005] — Maximum Value Extraction Attack Chain

Severity	**High**
Type	Logic
Target	`Multiple: Oracle + Liquidation + Keeper Reward`
Status	**Acknowledged (Acceptable Risk)**

Description

This is a multi-step exploit chain combining BIT-MEGA-003 (keeper reward on insolvent positions) and BIT-MEGA-004 (oracle manipulation). An attacker who can manipulate the oracle could (1) push prices to trigger mass liquidations, (2) act as the keeper to collect 5% rewards on each liquidation, and (3) reverse the price manipulation.

Impact

Theoretical extraction of 5% of all liquidated margin in a single attack, plus any directional trading profit from the artificial price movement.

Recommendations

- See recommendations for BIT-MEGA-003 and BIT-MEGA-004.
- Additionally: implement per-block and per-epoch keeper reward caps to bound extraction.

Client Response

> *Acknowledged. Components reclassified individually. Attack requires oracle manipulation of on-chain pool reserves, mitigated by absence of MEV infrastructure on Bittensor L1 and maxSlippage caps.*

[BIT-MEGA-006] — Protocol Insolvency Attack Chain

Severity	**High**
Type	Logic
Target	`Multiple: Interest + Exposure + Keeper Reward`
Status	**Resolved**

Description

A multi-step attack chain combining BIT-MEGA-001 (interest excluded from margin ratio), BIT-MEGA-002 (vault exposure asymmetry), and BIT-MEGA-003 (keeper reward drain). Together, these vulnerabilities could drive the protocol toward insolvency.

Impact

Potential total loss of vault reserves and insurance fund.

Client Response

> *Largely mitigated. BIT-MEGA-001 and BIT-MEGA-002 are fixed. BIT-MEGA-003 is reclassified as acceptable risk. Bad debt auto-pause provides a backstop.*

Medium

[BIT-MEGA-007] — No Admin Function Timelock

Severity	**Medium**
Type	Centralization
Target	`Admin.sol — all admin functions`
Status	**Acknowledged (Acceptable Risk)**

Description

No on-chain timelock exists for admin functions including `upgradeToAndCall`, `addMarket`, `pause`, and risk parameter changes. An attacker who compromises the multisig could execute arbitrary changes immediately.

Recommendations

- **Short term:** Implement a 24-48 hour timelock for `upgradeToAndCall` and parameter changes that affect user funds.
- **Long term:** Transition to a governance model with on-chain voting for critical parameter changes.

Client Response

> *Acknowledged. Protocol uses a multisig for admin roles. The multisig IS the delay mechanism. Every major DeFi protocol launched without a timelock initially (Uniswap, Aave, Compound all added timelocks post-launch). Nice-to-have improvement, not a deployment blocker.*

[BIT-MEGA-008] — Single Oracle Source with No Fallback

Severity	**Medium**
Type	Oracle Manipulation
Target	`Storage.sol → priceOracle; all price consumers`
Status	**Acknowledged (Acceptable Risk)**

Description

The protocol depends entirely on a single oracle (`priceOracle.calculatePriceFromReserves`). If this oracle malfunctions, returns incorrect data, or is manipulated, no fallback mechanism exists.

Client Response

> *Acknowledged. The oracle derives prices from Bittensor subnet TAO/Alpha pool reserves. This is the canonical on-chain price source — no alternative could provide the same data.*

[BIT-MEGA-009] — Oracle Front-Running via Price Observation

Severity	**Medium**
Type	Front-running / MEV
Target	`Oracle price derivation from pool reserves`

Status	**Acknowledged (Acceptable Risk)**

Description

Since oracle prices are derived from current pool state, any pending large swap in the mempool reveals the post-execution price. A sophisticated actor could front-run these swaps by opening positions before the price moves and closing after.

Client Response

> *Acknowledged. No MEV relays, PBS, or sophisticated searcher ecosystem exists on Bittensor L1. maxSlippage (capped at 10%) limits extractable value.*

[BIT-MEGA-010] — Insurance Fund Not Isolated Per Market

Severity	**Medium**
Type	Logic
Target	`VaultLib.sol → handleBadDebt()`
Status	**Acknowledged (Acceptable Risk)**

Description

The insurance fund is shared across all markets. Bad debt from one market can deplete the insurance fund, leaving other markets unprotected.

Client Response

> *Acknowledged. Shared insurance is a deliberate design choice. Per-market fragmentation means each market must be self-sufficient, leading to higher capital requirements and inefficient allocation. Bad debt auto-pause provides a backstop when badDebtOutstanding exceeds configurable threshold.*

[BIT-MEGA-011] — Bad Debt Socialization to Vault Depositors

Severity	**Medium**
Type	Logic
Target	`VaultLib.sol → handleBadDebt(); settleVaultPnl()`
Status	**Acknowledged (Acceptable Risk)**

Description

When the insurance fund is depleted, bad debt is absorbed by vault reserves, directly reducing the value available to vault depositors.

Client Response

> *Acknowledged. Bad debt tracking exists (badDebtOutstanding). Bad debt auto-pause provides a backstop. Standard design pattern across perpetual DEXs.*

[BIT-MEGA-012] — No Partial Liquidation Cooldown

Severity	**Medium**
Type	Denial of Service
Target	`Liquidation.sol → liquidatePosition()`
Status	**Acknowledged (Acceptable Risk)**

Description

No cooldown exists between partial liquidations of the same position. A keeper could repeatedly partially liquidate a position across consecutive blocks, extracting 5% keeper fee each time.

Client Response

> *Acknowledged. Rate-limiting liquidations would be actively harmful. Fastest possible liquidation is always optimal. Maintenance margin at 20% provides buffer.*

Low

[BIT-MEGA-013] — Funding Rate Bounded Manipulation

Severity	**Low**
Type	Oracle Manipulation
Target	`FundingLib.sol:83-87 → calculateFundingRate()`
Status	**Acknowledged (Acceptable Risk)**

Description

An attacker holding shorts can add longs to amplify the funding rate charged to OTHER longs. The gain is bounded by: (1) MAX_FUNDING_RATE cap at 75 bps/8h, (2) maxPositionSize limits, (3) trading fees of ~20 bps round trip, (4) price risk on the attacker's long.

Impact

Practical extractable value is small relative to required capital. Attack is economically unprofitable at current parameter settings.

Client Response

> *Acknowledged. Funding rate IS capped at MAX_FUNDING_RATE = 75 (0.75% per 8h interval). Multiple bounds constrain extraction.*

[BIT-MEGA-014] — Utilization Metric Excludes Unrealized PnL

Severity	**Low**
Type	Logic
Target	`VaultOperationsLib.sol → circuit breaker utilization`
Status	**Acknowledged (Acceptable Risk)**

Description

Circuit breaker utilization is calculated as `(totalReserves - availableReserves) / totalReserves` which measures collateral lock-up, not risk-adjusted exposure. Unrealized PnL doesn't feed into circuit breakers. However, no attacker can extract value from this — it's a capital efficiency concern, not an exploitable vulnerability.

Client Response

> *Acknowledged. The protocol uses three utilization formulas for different purposes. The gap is accepted as a design trade-off.*

Informational

[BIT-MEGA-015] — Stale Price Rejection

Severity	Informational
Type	Data Validation
Target	`Oracle price validation`
Status	Resolved

Description

Oracle now includes staleness protection. Prices older than MAX_PRICE_AGE are rejected. Defense-in-depth.

[BIT-MEGA-016] — Oracle Heartbeat Check

Severity	Informational
Type	Data Validation
Target	`Oracle price validation`
Status	Resolved

Description

Protocol rejects stale prices via heartbeat validation. Defense-in-depth measure complementing BIT-MEGA-015.

[BIT-MEGA-017] — Funding Update Race Condition

Severity	Informational
Type	Logic
Target	`Liquidation.sol:117 → updateFundingIndex()`
Status	Resolved

Description

`Liquidation.sol:117` calls `updateFundingIndex(market)` before checking liquidation status, ensuring fresh funding data is used in margin ratio calculations.

[BIT-MEGA-018] — Concurrent Liquidation Double-Spend

Severity	Informational
Type	Reentrancy
Target	`Liquidation.sol → liquidatePosition()`
Status	Resolved

Description

`nonReentrant` modifier prevents same-tx reentrancy. Cross-tx race condition is inherently prevented by EVM's single-threaded execution model.

[BIT-MEGA-019] — Interest Forces Premature Insolvency

Severity	Informational
Type	Arithmetic
Target	`LiquidationLib.sol → calculateMarginRatio()`
Status	Resolved

Description

Resolved by BIT-MEGA-001 fix. Interest is now included in margin ratio.

[BIT-MEGA-020] — Double Exposure Tracking on P2P

Severity	Informational
Type	Logic
Target	`VaultLib.sol → adjustExposureMappingsOnReduce()`
Status	Resolved

Description

Resolved by BIT-MEGA-002 fix. Clamped reduction prevents double-counting from P2P fills.

[BIT-MEGA-021] — Insurance Fund Bypass Path

Severity	Informational
Type	Logic
Target	`VaultLib.sol → handleBadDebt(), settleVaultPnl()`
Status	Resolved

Description

Bad debt handling correctly prioritizes: insurance fund first, then reserves, then tracks outstanding bad debt. No bypass path exists.

[BIT-MEGA-022] — Circuit Breaker Threshold Configuration

Severity	Informational
Type	Logic
Target	`VaultOperationsLib.sol → checkAndTriggerCircuitBreaker()`
Status	Resolved

Description

Circuit breakers implemented at three levels: global pause, per-market pause, and bad debt threshold pause. Configuration is adequate.

[BIT-MEGA-023] — Dynamic Vault Capacity Calculation

Severity	Informational
Type	Logic
Target	`VaultLib.sol → calculateDynamicCapacity()`
Status	**Resolved**

Description

VaultLib.calculateDynamicCapacity correctly implements utilization-based capacity reduction, reserve buffers, and volatility adjustments.

[BIT-MEGA-024] — Missing Vault Withdrawal Function

Severity	Informational
Type	Logic
Target	`Vault.sol:475 → withdrawFromVault()`
Status	**Resolved**

Description

Vault.withdrawFromVault implements admin vault withdrawal with safety checks: proportional withdrawal, reserve requirements, exposure-based collateral checks.

[BIT-MEGA-025] — Missing Fee Withdrawal Mechanism

Severity	Informational
Type	Logic
Target	`Admin.sol:332 → withdrawTradingFees()`
Status	**Resolved**

Description

Admin.withdrawTradingFees allows TREASURY_ROLE to withdraw accumulated trading fees. Correctly zeroes accumulatedTradingFees before transfer.

[BIT-MEGA-026] — Native Transfer Reentrancy

Severity	Informational
Type	Reentrancy
Target	`Functions using sendValue for native TAO transfers`

Status	**Resolved**

Description

All functions using sendValue for native TAO transfers are protected by the nonReentrant modifier from OpenZeppelin's ReentrancyGuardUpgradeable.

[BIT-MEGA-027] — Unchecked Fee Deduction Underflow

Severity	**Informational**
Type	Arithmetic
Target	`Position.sol:389 → fee deduction`
Status	**Resolved**

Description

Fee deduction checks marginBalances[trader] >= mem.closingFee before deducting. Insufficient balance causes fee to be skipped rather than underflow. Solidity 0.8.28 built-in overflow protection provides additional safety.

Section 8 — Gas Optimization Findings

No significant gas optimization opportunities were identified. The protocol already uses:

- Custom errors instead of revert strings
- Efficient storage packing via ERC-7201 namespaced storage
- Optimizer with 25 runs (size-optimized for 24KB limit compliance)
- Library pattern to offload computation from the proxy target

Section 9 — Fix Review / Remediation Log

After the initial findings delivery, the MegaTao team implemented fixes and Bitsec verified each remediation. Additionally, 38 raw agent findings were triaged with the MegaTao team, resulting in the 27 consolidated findings in this report.

Finding ID	Severity	Fix Commit	Status	Notes
BIT-MEGA-001	Critical	ca039bb	Resolved	Interest included in margin ratio calculation
BIT-MEGA-002	Critical	ca039bb	Resolved	Exposure reduction clamped to directional exposure
BIT-MEGA-003	High	–	Acknowledged	Keeper incentive design trade-off
BIT-MEGA-004	High	–	Acknowledged	No alternative oracle; no MEV on Bittensor L1
BIT-MEGA-005	High	–	Acknowledged	Depends on 003 + 004; residual risk accepted
BIT-MEGA-006	High	ca039bb	Resolved	Root causes (001, 002) fixed; 003 acknowledged
BIT-MEGA-007	Medium	–	Acknowledged	Multisig provides operational delay
BIT-MEGA-008	Medium	–	Acknowledged	No alternative oracle exists
BIT-MEGA-009	Medium	–	Acknowledged	No MEV infrastructure on Bittensor L1
BIT-MEGA-010	Medium	–	Acknowledged	Shared insurance deliberate; bad debt auto-pause
BIT-MEGA-011	Medium	–	Acknowledged	Standard design pattern; bad debt tracking exists
BIT-MEGA-012	Medium	–	Acknowledged	Cooldowns would increase bad debt during crashes
BIT-MEGA-013	Low	–	Acknowledged	Bounded by rate cap and trading fees
BIT-MEGA-014	Low	–	Acknowledged	Capital efficiency concern, not exploitable
BIT-MEGA-015	Info	9e473a7	Resolved	Defense-in-depth staleness check
BIT-MEGA-016	Info	9e473a7	Resolved	Heartbeat validation
BIT-MEGA-017	Info	9e473a7	Resolved	Funding updated before liquidation check
BIT-MEGA-018	Info	9e473a7	Resolved	nonReentrant + atomic state updates
BIT-MEGA-019	Info	ca039bb	Resolved	Resolved by BIT-MEGA-001 fix
BIT-MEGA-020	Info	ca039bb	Resolved	Resolved by BIT-MEGA-002 fix
BIT-MEGA-021	Info	9e473a7	Resolved	Insurance fund priority correct
BIT-MEGA-022	Info	9e473a7	Resolved	Three-level circuit breakers implemented
BIT-MEGA-023	Info	9e473a7	Resolved	Dynamic capacity correctly computed
BIT-MEGA-024	Info	9e473a7	Resolved	Vault withdrawal with safety checks
BIT-MEGA-025	Info	9e473a7	Resolved	Fee withdrawal for TREASURY_ROLE
BIT-MEGA-026	Info	9e473a7	Resolved	nonReentrant on all native transfers
BIT-MEGA-027	Info	9e473a7	Resolved	Balance check before fee deduction

Reclassified Findings (excluded from report)

The following 11 raw agent findings were reclassified as "Not Vulnerable" during triage:

Original ID	Claim	Reclassification Reason
VULN-277	Same-block liquidation possible	Intentional design; confirmed by explicit test. Delay creates bad debt.
VULN-284	Reserve unlocking race condition	EVM executes synchronously; no race conditions possible.
VULN-298	Wrong utilization metric	Design trade-off, not exploitable. Three formulas for different purposes.
VULN-924	Funding rate manipulation	Rate IS bounded by MAX_FUNDING_RATE cap (75 bps/8h).
VULN-930	Position limit bypass	Incorrect claim; total position enforcement exists in _updatePosition().
+ 6 others	Various	Duplicate findings, incorrect code assumptions, or non-applicable attack paths

Section 10 — Appendices

Appendix A — Vulnerability Type Taxonomy

Type	Description
Access Control	Authorization failures, missing role checks
Arithmetic	Overflow, underflow, precision loss, rounding errors
Reentrancy	Cross-function or cross-contract reentrancy
Logic	Incorrect business logic, off-by-one, state machine errors
Oracle Manipulation	Price manipulation, TWAP attacks, stale data
Front-running / MEV	Ordering attacks, sandwich attacks
Denial of Service	Gas griefing, unbounded loops, forced reverts
Data Validation	Missing input validation, type confusion
Cryptography	Signature malleability, weak randomness
Centralization	Privileged actor risk, admin key risk
Undefined Behavior	EVM edge cases, compiler-specific behavior
Informational	Code quality, best practice deviations

Appendix B — Severity Definitions

Severity	Definition
Critical	Direct loss of user funds or complete protocol takeover with no prerequisite conditions
High	Significant loss of funds or severe disruption; requires specific conditions or non-trivial attacker capability
Medium	Partial loss of funds, griefing, or material disruption; limited attack surface or requires user action
Low	Minor impact; unlikely exploit path or low severity consequence
Informational	No direct security risk; code quality, gas efficiency, or best practice observations

Appendix C — Scope File Manifest

In-scope Solidity files (41 files, 9,520 LOC):

Directory	Files	LOC
src/modules/	11 core + 8 wrappers	4,991 + 120
src/libraries/	12 files	3,515
src/storage/	1 file	616
src/implementations/	6 files	143
src/ (root)	2 files (Futures.sol, FuturesProxy.sol)	98
src/utils/	1 file	37
Total	**41 files**	**9,520**

Commit: `9e473a7fa070a39650ed74250b915ec75145f1d9`

Compiler: solc 0.8.28, optimizer enabled (25 runs), EVM target: london

Appendix D — Residual Risk Summary

The following risks are acknowledged by the MegaTao team and accepted as design trade-offs:

1. Oracle manipulation via pool reserves

If MEV infrastructure (flashbots, MEV relays, PBS) develops on Bittensor L1, the on-chain pool oracle could be manipulated via large swaps. This would enable flash crash liquidations, oracle front-running, and the maximum value extraction attack chain.

Mitigation: maxSlippage cap (10%), circuit breakers, bad debt auto-pause.

Recommendation: Monitor Bittensor ecosystem development and implement oracle bounds proactively.

2. Keeper reward amplification of bad debt

During mass insolvency events, the 5% keeper fee increases bad debt per liquidation.

Mitigation: Insurance fund absorbs first; bad debt auto-pause triggers at configurable threshold.

Recommendation: Consider a tiered reward model post-launch based on observed liquidation patterns.

3. Admin key risk without timelock

Multisig compromise enables immediate protocol changes including implementation upgrades.

Mitigation: Multisig requires multiple key compromise.

Recommendation: Implement on-chain timelock post-launch.

This report was generated through Bitsec's AI agent competition methodology on Subnet 60, with all findings validated and triaged by a senior human auditor. The competitive agent structure produced 38 raw findings across 9 agents, consolidated into 27 unique findings after deduplication and triage. All critical and high findings have been resolved or acknowledged with

documented risk acceptance rationale.

Report version: v1.0-final | Delivered: 2026-02-26